Java-Based Database Load Balancer Simulator

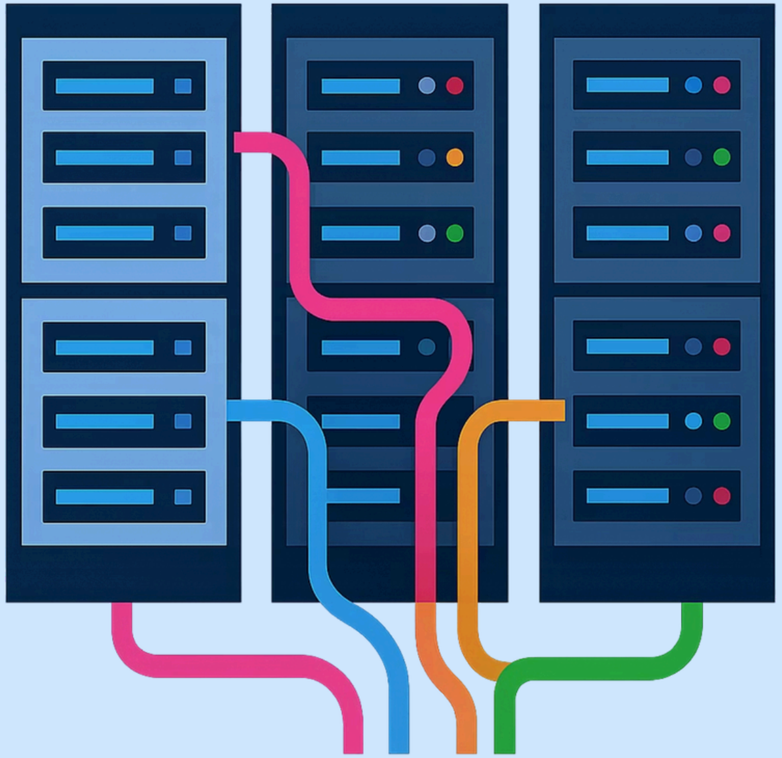
Using Spring Boot

Student Details

- Ananya sharma - 1032240169
- Grisha Jadhav - 1032240155
- Dhruva Tikhe - 1032240158
- Gaurika Nawani - 1032240181

Project Summary

A Java 17 and Spring Boot 3.2 application that simulates intelligent query routing across three MySQL instances, featuring multiple load balancing algorithms, distributed transaction handling, and a real-time web dashboard.



Problem Statement

Traditional single-node database architectures are ill-equipped for modern high-concurrency workloads, introducing several critical limitations:

Single Point of Failure

Any crash or outage of the sole database node brings the entire application to a halt, with no fallback or redundancy mechanism available.

Poor Load Distribution

All queries — read and write — funnel through one node, creating bottlenecks and significantly degrading throughput under concurrent traffic.

High Latency

Under heavy traffic, query queue depth increases rapidly, leading to unacceptable response times and poor user experience.

No Real-Time Visibility

Administrators lack application-level insight into node health, active connection counts, or query routing decisions as they happen.

Project Objectives

This project targets a clearly defined set of engineering goals spanning algorithm design, data integrity, and observability.

1 **Design a Database-Level Load Balancer**

Build an application-layer routing engine that intercepts queries and distributes them intelligently across multiple MySQL instances.

2 **Implement Multiple Routing Algorithms**

- Round Robin – sequential, equal distribution
- Least Connections – dynamic, load-aware routing
- Weighted Round Robin – capacity-proportional routing

3 **Ensure Data Consistency**

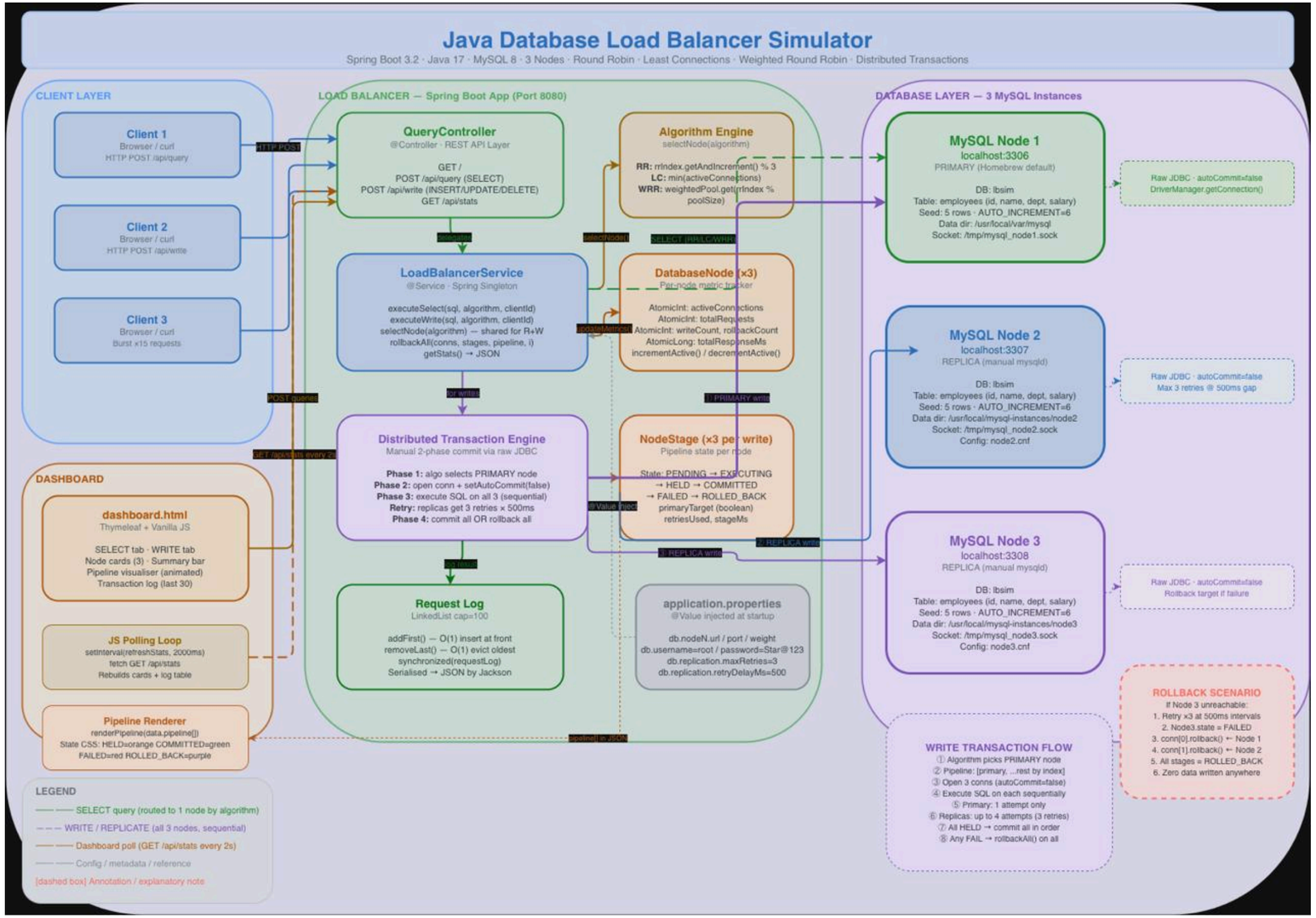
Propagate write operations to all nodes using a manual two-phase commit strategy, with retry logic and full rollback on failure.

4 **Real-Time Monitoring Dashboard**

Deliver a Thymeleaf-powered web UI that shows live node metrics, active connections, and current algorithm selection.

System Architecture

The system is structured in clearly separated layers — from the client-facing web dashboard down to the individual MySQL instances — enabling modular development and testability.



Key Architectural Decisions

- Spring Boot 3.2 as the application container
- Three isolated MySQL instances on distinct ports
- JDBC used directly — no ORM layer — for precise connection control

Layer Responsibilities

- Controller layer exposes REST endpoints for query submission
- Service layer encapsulates all routing and transaction logic
- Dashboard polls metrics in real-time via lightweight REST calls

Tools & Technologies

Java 17
LTS release providing modern language features, enhanced switch expressions, and improved performance.

Spring Boot 3.2
Rapid application framework supplying REST controllers, dependency injection, and embedded Tomcat server.

MySQL (×3)
Three independent instances on ports 3306, 3307, and 3308 simulating a distributed database cluster.

JDBC
Direct Java Database Connectivity for fine-grained, manual control over connections and transactions.

Thymeleaf
Server-side Java template engine rendering the real-time monitoring dashboard as HTML pages.

Maven + macOS
Maven manages dependencies and build lifecycle; macOS with Homebrew used to provision MySQL instances locally.

Modules Overview

The application is decomposed into five cohesive modules, each with a single, well-defined responsibility.

Query Controller

Exposes REST endpoints that accept incoming SQL queries from clients, validates input, and delegates routing decisions to the Load Balancer Service.

Load Balancer Service

The core routing engine. Selects the appropriate database node based on the active algorithm (Round Robin, Least Connections, or Weighted Round Robin) and dispatches the query.

Database Node & Transaction Manager

Each node object tracks active connections and health status. The Distributed Transaction Manager orchestrates multi-node writes using a manual two-phase commit with retry and rollback.

Load Balancing Algorithms

Three distinct routing strategies are implemented and switchable at runtime via the dashboard — each suited to a different traffic pattern.

1

Round Robin

Queries are dispatched to nodes in a fixed, sequential cycle: Node 1 → Node 2 → Node 3 → repeat. Ideal for homogeneous nodes with equal capacity. Simple, predictable, and low-overhead.

2

Least Connections

At dispatch time, the balancer inspects the active connection count on each node and routes the query to the least-loaded instance. Adapts dynamically to real-world query duration variability.

3

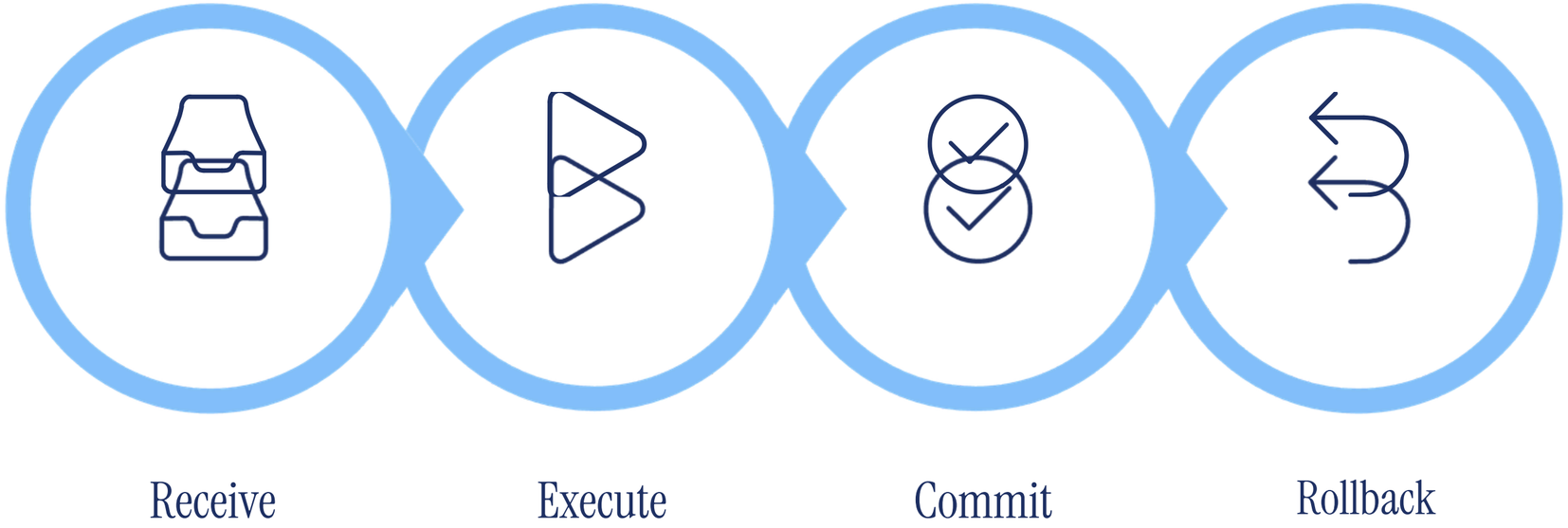
Weighted Round Robin

Each node is assigned a weight reflecting its relative capacity. Nodes with higher weights receive proportionally more queries per cycle, enabling heterogeneous cluster management.

All three algorithms are implemented as strategy-pattern classes and can be hot-swapped via a single dashboard toggle without restarting the application.

Distributed Transaction Handling

Write operations require consistency across all three nodes. A manual two-phase commit protocol is implemented entirely in application code over raw JDBC connections.



Protocol Details

- Auto-commit disabled on all connections before execution
- Write query executed concurrently on all three nodes
- Commit issued only if all executions succeed
- Any single failure triggers full rollback across all nodes

Resilience Mechanisms

- Retry mechanism: up to 3 attempts per failed node
- Retry delay: 500 ms between each attempt
- Connections returned to pool state after rollback
- Error details logged with node ID and failure reason

Use Cases & Challenges

Practical Use Cases

Enterprise Load Distribution

Mirrors real-world patterns in high-traffic enterprise applications where a single database node cannot sustain peak query throughput.

High Availability Systems

Demonstrates how application-layer routing can mask node failures and maintain availability without external hardware load balancers.

Distributed Systems Education

Provides a fully observable, local simulation environment for studying distributed transaction coordination and consistency models.

Engineering Challenges

Concurrent Request Handling

Managing thread safety when multiple requests simultaneously read and update node connection counters required careful synchronisation.

Data Consistency Across Nodes

Ensuring all three nodes reflect the same committed state without a distributed transaction coordinator framework demanded precise manual protocol design.

Manual JDBC Connection Management

Without connection pooling, tracking, reusing, and closing connections reliably — especially on rollback paths — required rigorous resource handling.

Future Scope & Conclusion

Future Enhancements

01

HikariCP Connection Pooling

Replace manual JDBC connection management with an enterprise-grade pool to improve throughput and reduce connection overhead.

02

Cloud Deployment

Containerise the application with Docker and deploy to AWS or GCP to test the balancer against real distributed network latency.

03

AI-Based Load Prediction

Train a lightweight ML model on historical query metrics to proactively rebalance traffic before saturation occurs.

04

Advanced Real-Time Analytics

Integrate a time-series dashboard (e.g. Grafana + Prometheus) for deeper observability into per-node performance trends.

Conclusion

- Successfully designed and implemented a database-level load balancer entirely in application code using Java 17 and Spring Boot 3.2
- Three routing algorithms — Round Robin, Least Connections, and Weighted Round Robin — proven functional under simulated load
- Distributed transaction handling achieved via a manual two-phase commit with retry logic and full rollback guarantees
- Real-time monitoring dashboard provides live node metrics and runtime algorithm switching
- Project demonstrates core distributed systems principles within a fully observable, locally